TD N°1 : Techniques de conteneurisation et micro-services

Objectifs pédagogiques

À la fin de ce TD, l'étudiant doit être capable de :

- Comprendre l'architecture micro-services.
- Expliquer le rôle de **Docker.**
- Utiliser **Docker Compose** pour un environnement multi-services

Partie Théorique :

Partie 1 — Questions de compréhension

1. Définir l'architecture micro-services.
2. Citer trois avantages des micro-services.
3. Quelle différence existe-t-il entre micro-services et architecture monolithique ?
4. Pourquoi la conteneurisation est-elle importante pour les micro-services ?
5. Quels sont les avantages des conteneurs par rapport aux machines virtuelles ?

Partie 2 — Docker

1. Expliquer le rôle de Docker dans une architecture micro-services.
2. Que contient un conteneur ?
3. Quelle est la différence entre :
    - Image Docker
    - Conteneur Docker
4. Donner les avantages de Docker pour :
    - Portabilité
    - Reproductibilité
    - Déploiement

Partie 3 — Docker Compose

1. Pourquoi utiliser Docker Compose ?
2. Donner un exemple d'application nécessitant plusieurs conteneurs.

3. Que permet de simuler Docker Compose ?

Partie 4 — Kubernetes

1. Pourquoi Docker seul ne suffit pas en production ?
2. Définir :
    - Pod
    - Deployment
    - Service
    - Persistance claims volume (pvc)
    - Secret
    - Config map
3. Expliquer :
    - Rolling update
    - Auto-scaling
    - Haute disponibilité

Partie 5 — Questions avancées :

1. Différence Docker vs Kubernetes
2. Monolithe modulaire vs micro-services
3. Problèmes des micro-services (latence, complexité…)
4. Observabilité (logs, metrics, tracing)
5. CI/CD et micros-ervices

Partie Pratique :

Partie 6 — Mini-projet Master : Architecture micro-services cloud-native avec Spring Boot

Objectif :

Concevoir, développer et déployer une application **e-commerce micro-services** en utilisant :

- **Spring Boot** pour les micro-services
- **Docker** pour la conteneurisation
- **Docker Compose** pour l'environnement local

Approche attendue : **architecture cloud-native professionnelle**.

1. **Préparation environnement (obligatoire — évaluable)**

- **Installation outils**

- Installer IntelliJ IDEA
- Installer JDK 17 ou 21
- Installer Maven
- Configurer variables d’environnement :

JAVA_HOME
PATH

Vérification :

```
java -version
mvn -v
```

- **Génération projets**

Créer les projets via **Spring Initializr** :

Services :

- catalogue-service
- commande-service
- paiement-service
- gateway-service
- discovery-service
- config-service

Dépendances :

- Spring Web
- Spring Data JPA
- Lombok
- Actuator
- OpenFeign
- Eureka Client
- Config Client

2. **Développement attendu :**

   Chaque micro-service doit respecter l’architecture hexagonale :

- controller
- service

- domain
- repository
- dto

Exigences :

- REST API versionnée
- Gestion erreurs globale
- Logging structuré
- Validation
- DTO mapping
- Tests unitaires

Communication :

- OpenFeign obligatoire

3. **Installation Docker**

Thème :

Mise en place de l’environnement de conteneurisation.

Tâches :

- Installer Docker Desktop
- Vérifier que Docker fonctionne
- Vérifier le moteur de conteneurs

Commandes de vérification :

```
docker --version
docker info
docker run hello-world
```

4. **Création Dockerfile pour chaque micro-service :**

Pour chaque micro-service :

Compiler le projet :

mvn clean package

Créer un fichier Dockerfile à la racine du service.

Définir :

- Image Java
- Copie du jar
- Port

- Commande run

5. **Construction des images à travers la ligne de commande :**

Depuis chaque micro-service :

Construire les images :

```
docker build -t catalogue-service:1.0 .
docker build -t commande-service:1.0 .
docker build -t paiement-service:1.0 .
docker build -t gateway-service:1.0 .
docker build -t discovery-service:1.0 .
docker build -t config-service:1.0 .
```

Lister les images :

```
docker images
```

6. **Lancer les conteneurs à travers localhost :**

➢ Lancer les conteneurs :

```
docker run -p 8081:8080 catalogue-service:1.0
docker run -p 8082:8080 commande-service:1.0
docker run -p 8083:8080 paiement-service:1.0
docker run -p 8084:8080 gateway-service:1.0
docker run -p 8761:8761 discovery-service:1.0
docker run -p 8888:8888 config-service:1.0
```

➢ Vérifier :

```
docker ps
```

- Tester :
- http://localhost:8081
- http://localhost:8761
- http://localhost:8888

7. **Intégration de Docker Compose :**

But : Orchestration locale complète des microservices Spring Boot avec **Docker Compose**.

Partie A — Vérification de l'environnement

◆ **Tâche 1 : Vérifier l'installation**

- Vérifier que **Docker Desktop** est installé.
- Vérifier que Docker Compose est disponible.

Commande :

```
docker compose version
```

Partie B — Création du fichier docker-compose.yml

◆ **Tâche 2 : Créer le fichier**

- Se placer dans le dossier racine du projet.
- Créer un fichier :

docker-compose.yml

🔹 Tâche 3 : Définir les services

Dans le fichier, déclarer :

- discovery-service
- config-service
- catalogue-service
- commande-service
- paiement-service
- gateway-service

Partie C — Configuration des services

🔹 Tâche 4 : Configurer les images

Pour chaque service :

- Référencer l'image Docker construite précédemment.
- Mapper les ports (host:container).
- Ajouter depends_on pour gérer l'ordre logique de démarrage.

🔹 Tâche 5 : Ajouter les variables d'environnement

Pour chaque micro-service :

- Définir :
  - SPRING_PROFILES_ACTIVE=docker
  - URL du Config Server
  - URL Eureka

🔹 Tâche 6 : Configurer le réseau interne

- Utiliser le réseau par défaut Docker Compose.
- Vérifier que les services communiquent via leur nom (ex : http://discovery-service:8761).

Partie D — Lancement de l'architecture

🔹 Tâche 7 : Démarrer tous les services

Commande :
docker compose up
ou en arrière-plan :
docker compose up -d

🔹 Tâche 8 : Vérifier le bon fonctionnement

- Lister les conteneurs actifs :

docker compose ps

- Vérifier l'accès via navigateur :
  - http://localhost:8761
  - http://localhost:8888
  - http://localhost:8084

Partie E — Tests et validation

🔹 Tâche 9 : Vérifier la communication inter-services

- Tester qu'un microservice s'enregistre correctement dans Eureka.
- Tester les appels via la Gateway.

🔹 Tâche 10 : Consulter les logs

Commande :
docker compose logs -f

Partie F — Arrêt de l’environnement

🔹 **Tâche 11 : Arrêter les services**

docker compose down

🔹 **Tâche 12 : Ajouter les bases de données**

- Ajouter un service PostgreSQL ou MySQL par microservice.
- Configurer les variables d’environnement DB.
- Ajouter des volumes persistants.

🔹 **Tâche 13 : Ajouter healthchecks**

- Définir un healthcheck pour chaque microservice.
- Vérifier que Docker Compose attend que le service soit healthy.

🔹 **Tâche 14 : Utiliser build au lieu de image (mode dev)**

- Modifier le compose pour construire automatiquement les images à partir des Dockerfiles.